

Video Warp Processor v1.1 LogiCORE IP Product Guide

Vivado Design Suite

PG396 (v1.1) July 9, 2025



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	7
Core Overview.....	8
Applications.....	8
Licensing and Ordering.....	8
Chapter 3: Product Specification.....	10
Standards.....	10
Performance.....	10
Resource Use.....	11
Port Descriptions.....	11
Register Space.....	14
Chapter 4: Designing with the Core.....	25
Clocking.....	25
System Considerations.....	25
Chapter 5: Design Flow Steps.....	27
Customizing and Generating the Core.....	27
Constraining the Core.....	30
Simulation.....	31
Synthesis and Implementation.....	31
Chapter 6: Example Design.....	32
Synthesizable Example Design.....	33
Appendix A: Verification, Compliance, and Interoperability.....	42
Appendix B: Debugging.....	43



Finding Help with AMD Adaptive Computing Solutions.....	43
Debug Tools.....	44
Hardware Debug.....	45
Appendix C: Application Software Development.....	46
Appendix D: Additional Resources and Legal Notices.....	49
Finding Additional Documentation.....	49
Support Resources.....	50
References.....	50
Revision History.....	50
Please Read: Important Legal Notices.....	51

Introduction

The AMD LogiCORE™ IP Video Warp Processor provides a video processing block that digitally manipulates input video frames to generate warped output video frames. It has a memory interface for input and output. This IP core supports keystone, pincushion, barrel, arbitrary, scaling, rotation, and translation distortions. This highly configurable IP core supports the ability of in-system programming through a comprehensive register interface to control it. A comprehensive set of interrupt status bits are provided to enable monitoring and control by the provided driver API.

Features

- Memory-mapped AXI4 interface.
- Keystone, pincushion, barrel and arbitrary distortions.
- 0.5 – 2.0 scaling.
- Up to 90° rotation.
- Translation up to the source image boundaries.
- Spatial resolutions from 320 × 240 up to 3840 × 2160.
- RGB and YUV 4:4:4 8-bit, 10-bit and 12-bit per color component.
- Dynamically configurable source and destination buffer addresses.
- Supports throughputs of up to 3840x2160p60, depending on the device family.
- 32-bit and 64-bit external memory address access.
- 32, 64, 128 AXI4 memory interface data widths.
- Bi-linear and bi-cubic filters for quality control.
- Interfaces with memories greater than 120 MB (minimum memory size required to hold input, output and remap vector of a 3840x2160 RGB 12-bit frame).
- BRAM lines support up to 128.
- URAM support enabled.

IP Facts

LogiCORE IP Facts Table	
Core Specifics	
Supported Device Family ¹	AMD UltraScale+™, AMD UltraScale™, 7 series FPGAs, AMD Zynq™ 7000 SoC AMD Versal™ adaptive SoC
Supported User Interfaces	AXI Master Lite, AXI4-Lite
Resources	Performance and Resource Use web page for Warp Filter IP Performance and Resource Use web page for Warp Initializer IP
Provided with Core	
Design Files	Not Provided
Example Design	Verilog
Test Bench	Not Provided
Constraints File	Xilinx Design Constraints (XDC)
Simulation Model	Encrypted RTL
Supported S/W Driver ²	Standalone
Tested Design Flows ³	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the <i>Vivado Design Suite User Guide: Release Notes, Installation, and Licensing</i> (UG973).
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: 76221
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

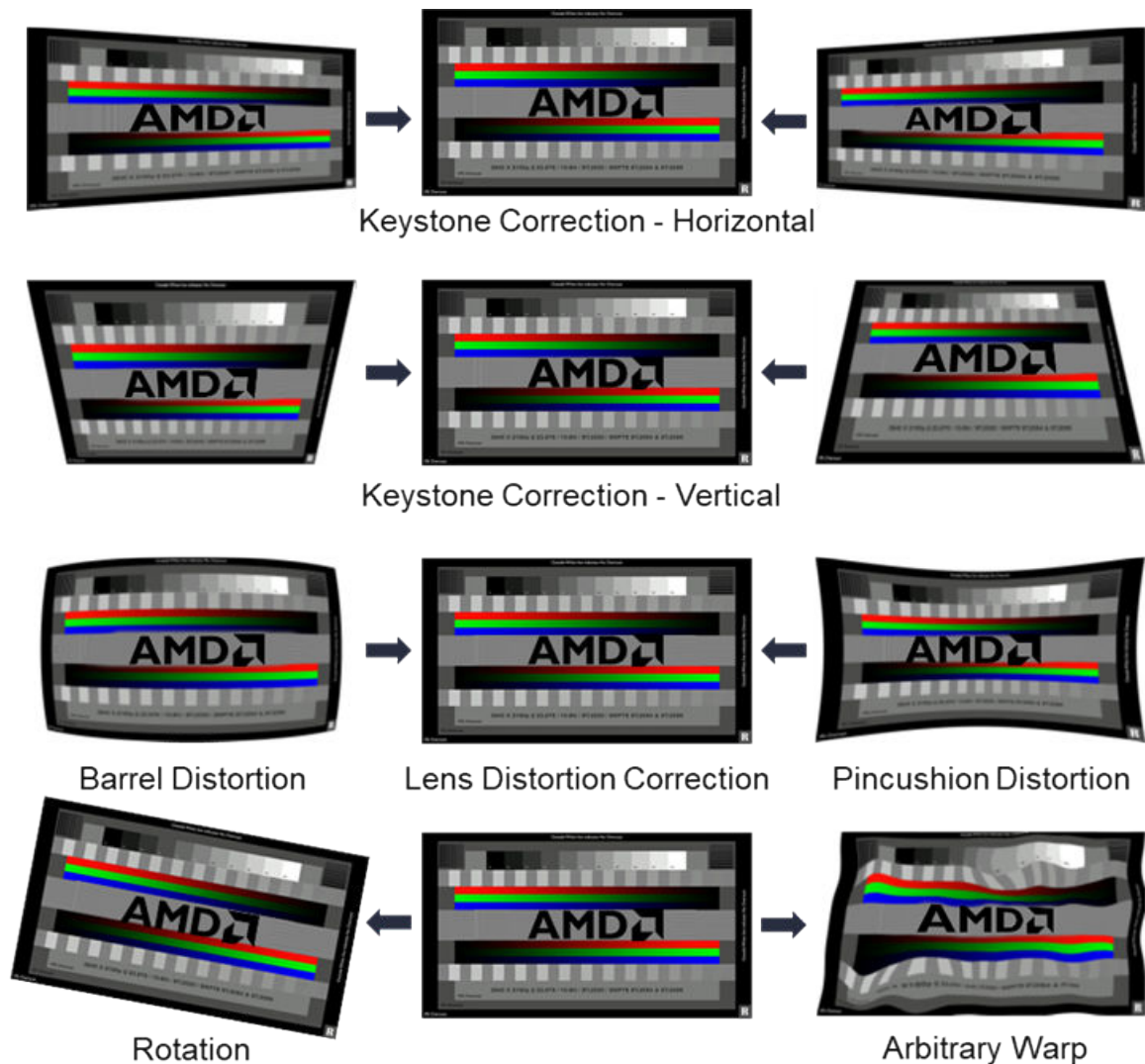
Notes:

1. For a complete list of supported devices, see the Vivado IP catalog.
2. Standalone driver details can be found in <install_directory>/Vitis/<release>/data/embeddedsw/doc/xilinx_drivers_api_toc.htm.
3. For the supported versions of third-party tools, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#)).

Overview

The Video Warp Processor supports real-time distortions types such as barrel, pincushion, keystone, as well as an arbitrary distortion mode. Image rotation, translation, and scaling are also supported by this core. The transformations available in this core allow the manipulation of real time video to accommodate distortions caused by optical transformations due to lenses, or physical transformations due to projections onto surfaces from angles, or irregular geometries.

Figure 1: Image and Video Transforms in the Video Warp Processor



The core is aimed at applications requiring image correction, such as automotive surround view cameras, medical endoscopy, video conferencing, and surveillance systems. It is also aimed at display applications, from projectors used in entertainment and simulation, to creative displays used in digital signage.

The Warp Processor consists of two IP blocks: the Warp Initializer and the Warp Filter. The Warp Initializer IP considers the warping parameters as input, and writes intermediate parameters (known as Warp Filter parameters) to the external memory. The Warp Filter IP reads the Warp Filter parameters and the image data from the external memory, and performs distortion interpolation to reconstruct the output image. The Warp Filter IP writes back the output image to the external memory controller.

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Register Space](#)
 - [Clocking](#)
 - [Customizing and Generating the Core](#)
 - [Chapter 6: Example Design](#)

Core Overview

The Warp Initializer and Warp Filter are independent, configurable IPs. However, the two modules work together to perform the complete warp distortion. They read the distortion parameters from a descriptor in external memory. Based on these distortion parameters, the Warp Initializer generates Warp Filter parameters, which are written into the external memory. After the Warp Filter parameters are written into the external memory, the Warp Initializer generates an interrupt. Whenever the video or warp distortion parameters are changed, the Warp Initializer must be reconfigured and run again before the Warp Filter is triggered to distort images.

The Warp Filter is a configurable core that reads the video distortion parameters and video parameters from the external memory descriptor along with the filter initialization parameters and source image data. The warp processor supports resolutions from 320×240 to 3840×2160, with up to three color components, each of 8, 10, or 12 bits. The Warp Filter reads the image from the source buffer address, using bilinear and bicubic interpolation to get a moderate and high quality warped image, which is written to the destination buffer address. The Warp Filter generates an interrupt after it completes writing the output. The source buffer and the destination buffer addresses can be dynamically changed so that you can point to any buffer address while the IP is running. The new buffer addresses are used from the next frame onwards.

Applications

The Video Warp Processor IP has a wide scope, and can be used in applications such as digital cinema, camera lens correction, virtual reality, automobile, projector, and embedded applications.

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the [End User License](#).

For more information about this core, visit the Video Warp Processor [product web page](#).

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

License Checkers

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- `write_bitstream` (Tcl command)



IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Product Specification

Standards

The Warp Processor is compliant with the AXI4-Lite interconnect and memory mapped AXI4 interface standards. For additional information, see the Video IP: AXI Feature Adoption section of *Vivado Design Suite: AXI Reference Guide* ([UG1037](#)).

Note: For information on getting video streams into and out of memory, as well as correct memory formatting, see the *Video Frame Buffer Read and Video Frame Buffer Write LogiCORE IP Product Guide* ([PG278](#)).

Performance

The following sections detail the performance characteristics of the Warp Processor.

Maximum Frequencies

The following are typical clock frequencies for the target devices:

Note: The speed grades mentioned are the minimum speed grades required to achieve 4 k at 60 FPS.

- AMD UltraScale+™ device with –1 speed grade or higher: 300 MHz
- AMD Kintex™ 7 and AMD Kintex™ UltraScale™ devices with –2 speed grade or higher: 300 MHz

The maximum achievable clock frequency can vary. The maximum achievable clock frequency and all resource counts can be affected by other tool options, additional logic in the device, using a different version of AMD tools, and other factors.

Table 1: Warp Initializer Latency

	Fixed Distortions (Resource Optimized)	Fixed Distortion (Latency Optimized)	With Arbitrary (Resource Optimized)
Warp Initializer latency	1.8 sec	236 msec	2.5 sec

Resource Use

For additional details about performance and resource use, see:

- [Performance and Resource Use web page for Warp Filter IP](#)
- [Performance and Resource Use web page for Warp Initializer IP](#)

Port Descriptions

The core interfaces are shown in the following figures.

Figure 2: Warp Initializer Core Ports

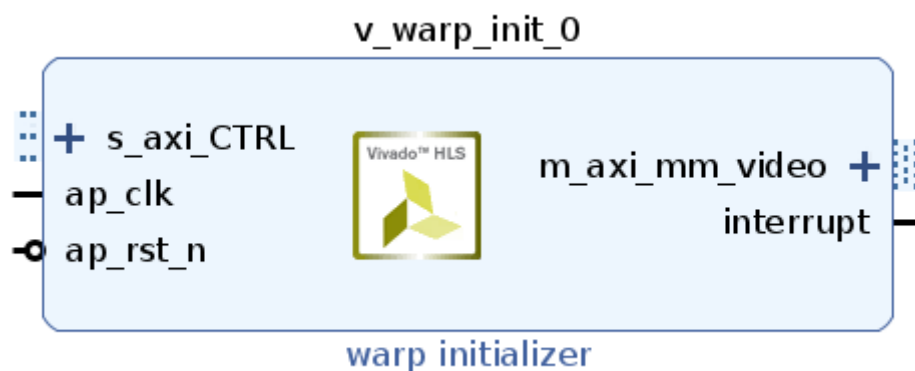


Figure 3: Warp Filter Core Ports



Common Interface Signals

Table 2: Common Interface Signals

Signal Name	I/O	Width	Description
ap_clk	I	1	Video core clock
ap_rst_n	I	1	Video core active-Low clock enable
interrupt	O	1	Interrupt Request Pin

The `ap_clk` and `ap_rst_n` signals are shared between the core, memory mapped AXI4 data interfaces, and the AXI4-Lite control interface.

- **ap_clk:** The memory mapped AXI4 and AXI4-Lite interfaces must be synchronous to the core clock signal `ap_clk`. All memory mapped AXI4 interface input signals and AXI4-Lite control interface input signals are sampled on the rising edge of `ap_clk`.
- **ap_rst_n:** The `ap_rst_n` pin is an active-Low, synchronous reset input pertaining to both AXI4-Lite and memory mapped AXI4 interfaces. When `ap_rst_n` is set to 0, the core resets at the next rising edge of `ap_clk`.
- **interrupt:** The interrupt status output bus can be integrated with an external interrupt controller that has independent interrupt enable mask, interrupt clear, and interrupt status registers that allow interrupt aggregation to the system processor.

AXI4-Lite Control Interface

The AXI4-Lite interface allows you to dynamically control the parameters within the core. The configuration can be accomplished using an AXI4-Lite master state machine or an embedded Arm® or soft system processor such as the MicroBlaze™ processor. The Warp Processor can be controlled through the AXI4-Lite interface using the driver functions provided in the AMD Vitis™ software platform. To understand how to create an application project in the AMD Vitis™ unified software platform and how to use driver APIs, refer to [Chapter 6: Example Design](#) and [Appendix C: Application Software Development](#). The following table shows the AXI4-Lite control interface signals.

This interface runs at the `ap_clk` clock.

Table 3: AXI4-Lite Control Interface Signals

Signal Name	I/O	Width	Description
s_axi_ctrl_aresetn	I	1	Reset
s_axi_ctrl_aclk	I	1	Clock
s_axi_ctrl_awaddr	I	18	Write Address
s_axi_ctrl_awprot	I	3	Write Address Protection
s_axi_ctrl_awvalid	I	1	Write Address Valid
s_axi_ctrl_awready	O	1	Write Address Ready
s_axi_ctrl_wdata	I	32	Write Data
s_axi_ctrl_wstrb	I	4	Write Data Strobe
s_axi_ctrl_wvalid	I	1	Write Data Valid
s_axi_ctrl_wready	O	1	Write Data Ready
s_axi_ctrl_bresp	O	2	Write Response
s_axi_ctrl_bvalid	O	1	Write Response Valid
s_axi_ctrl_bready	I	1	Write Response Ready
s_axi_ctrl_araddr	I	18	Read Address
s_axi_ctrl_arprot	I	3	Read Address Protection
s_axi_ctrl_arvalid	I	1	Read Address Valid
s_axi_ctrl_arready	O	1	Read Address Ready
s_axi_ctrl_rdata	O	32	Read Data
s_axi_ctrl_rresp	O	2	Read Data Response
s_axi_ctrl_rvalid	O	1	Read Data Valid
s_axi_ctrl_rready	1	1	Read Data Ready

Memory Mapped AXI4 Interface

There is a memory mapped AXI4 interface named `m_axi_mm_video` in the Warp Initializer. The Warp Filter can be configured with various levels of performance (for example, Performance Level 0 and Performance Level 1). In the Warp Filter, the number of memory mapped AXI4 interfaces changes based on the performance level selected in the IP Configuration tab. For Performance Level 0, only two memory mapped AXI4 interfaces, named `m_axi_mm_video_read` and `m_axi_mm_video_write`, are added to the Warp Filter. For Performance Level 1, two more memory mapped AXI4 interfaces named `m_axi_mm_video_read1` and `m_axi_mm_video_write1` are added to the Warp Filter. The memory mapped AXI4 interfaces run on the `ap_clk` clock domain. The signals follow the specification as defined in the *Vivado Design Suite: AXI Reference Guide* ([UG1037](#)).

Register Space

Register Address Space for Initializer IP

The Warp Initializer and Warp Filter have specific registers that allow you to control the operation of the core. All registers have an initial value of 0.

The following table provides a detailed description of all the registers that apply globally to the IP.

Table 4: Register Address Space for Warp Initializer

Address (hex) BASEADDR+	Register Name	Access Type	Register Description
0x0000	Control Signals	R/W	Bit[0] = <code>ap_start</code> (R/W/COH) ²
			Bit[1] = <code>ap_done</code> (R/COR) ³
			Bit[2] = <code>ap_idle</code> (R)
			Bit[3] = <code>ap_ready</code> (R)
			Bit[7] = <code>auto_restart</code> (R/W)
			Others = Reserved
0x00004	Global Interrupt Enable	R/W	Bit[0] = Global interrupt enable
			Others = Reserved
0x00008	IP Interrupt Enable	R/W	Bit[0] = <code>ap_done</code>
			Bit[1] = <code>ap_ready</code>
			Others = reserved
0x0000C	IP Interrupt Status register	R/TOW ¹	Bit[0] = <code>ap_done</code>
			Bit[1] = <code>ap_ready</code>
			Others = Reserved

Table 4: Register Address Space for Warp Initializer (cont'd)

Address (hex) BASEADDR+	Register Name	Access Type	Register Description
0x00010	Descriptor Address register 0	R/W	Bit[31] to Bit[0] = LSB 32 bits of the descriptor address
0x00014	Descriptor Address register 1	R/W	Bit[31] to Bit[0] = MSB 32 bits of the descriptor address
0x00020	MAXI Control Address register	R/W	Bit[31] to Bit[0] = Program 0 into this register.

Notes:

1. TOW – toggle on write.
2. COH – clear on handshake.
3. COR – clear on read.
4. The Control register (0x0000), Global Interrupt Enable register (0x0004), IP Interrupt Enable register (0x0008), and IP Interrupt Status register (0x000C) are explained in section [S_AXILITE Control Register Map](#) of UG1399. In UG1399, these register definitions might have some additional bits, but in this IP, only bits mentioned in this table are accessed. Consider only these bits when accessing the Control register, Global Interrupt Enable register, IP Interrupt Enable register, and IP Interrupt Status register.

Register Description

- **Control (0x00000) register:** This register controls the operation of the Warp Initializer. Bit[0] of the Control register, `ap_start`, kicks off the core from software. Writing 1 to this bit starts the core.

Bit[1] of the Control register, `ap_done`, indicates when the IP has completed all operations in the current transaction. A logic 1 on this signal indicates that the IP has completed all operations in this transaction.

Bit[2] of the Control register, `ap_idle`, indicates if the IP is operating or idle (no operation). The idle state is indicated by logic 1. This signal is asserted Low when the IP starts operating. This signal is asserted High when the IP completes operation and no further operations are performed.

Bit[3] of the Control register, `ap_ready`, indicates when the IP is ready for new inputs. It is set to logic 1 when the IP is ready to accept new inputs, indicating that all input reads for this transaction are completed. If the IP has no operations in the pipeline, new reads are not performed until the next transaction starts. This signal is used to make a decision on when to apply new values to the input ports and whether to start a new transaction. Bit[3] to Bit[6] are not used.

Bit[7] of the Control register, `auto_restart`, can be set to enable the auto-restart mode and then the IP restarts automatically at the end of each transaction.

- **Global Interrupt Enable (0x00004) register:** This register is the master control for all interrupts. Bit[0] can be used to enable or disable all core interrupts.

- **IP Interrupt Enable (0x00008) register:** This register allows interrupts to be enabled selectively. Currently, the following interrupt sources are available: `ap_done` and `ap_ready`. `ap_done` is triggered after the frame processing is complete, and `ap_ready` is triggered after the core is ready to start processing the next frame.
- **IP Interrupt Status (0x0000C) register:** This is a dual purpose register. When an interrupt occurs, the corresponding interrupt source bit is set in this register. In the readback mode (get status), the interrupting source can be determined. In the writeback mode (clear interrupt), the requested interrupt source bit is cleared.
- **IP Descriptor Address Register 0 (0x00010) Register:** This register allows to configure the LSB 32-bit of the 64-bit descriptor address to the IP.
- **IP Descriptor Address Register 1 (0x00014) Register:** This register allows to configure the MSB 32-bit of the 64-bit descriptor address to the IP.
- **IP MAXI Control Address (0x00020) Register :** This register is the Warp Initializer Control address. Program 0 into this register.

Warp Initializer IP Descriptor Structure

The Warp Initializer IP descriptor should be placed in external memory in the following format. The address of this descriptor is provided to the IP through the Descriptor Address register and the IP reads the descriptor from this address. The descriptor is a tightly packed structure of size 144 bytes in memory.

```
typedef struct {
    u16 width;
    u16 height;
    u16 bytes_per_pixel;
    u8 reserved_0[2];
    u64 filter_table_addr_0;
    u64 filter_table_addr_1;
    u32 width_Q4;
    u32 height_Q4;
    s16 k_pre;
    s16 k_post;
    u16 k0_pre;
    u16 k1_pre;
    u16 k0_post;
    u16 k1_post;
    u32 Q_fact_pre;
    u32 Q_fact_post;
    s8 k0_pre_Q_bits;
    s8 k0_post_Q_bits;
    s8 k1_pre_Q_bits;
    s8 k1_post_Q_bits;
    s16 h[6];
    s32 h_trans[3];
    u8 h_Qbits[6];
    u8 reserved_1[2];
    u32 normfactor;
    u16 cenX;
    u16 cenY;
    u32 num_ctrl_pts;
```



```

u64 src_ctrl_x_pts;
u64 src_ctrl_y_pts;
u64 src_tangents_x;
u64 src_tangents_y;
u64 interm_x;
u64 interm_y;
u8 warp_type;
u8 reserved_2[3];
u32 valid_segs;
u32 lblk_count;
u32 line_num_seg;
} XWarpInitVector_Hw;
S8 : Signed 8 bit
S16: Signed 16 bit
U8 : Unsigned 8 bit
U16: Unsigned 16 bit
U32: Unsigned 32 bit
U64: Unsigned 64 bit

```

Note: The IP does not support cameras with k1,k2, and k3 parameters. It only supports the K value to calculate K_pre and K_post. However, the newer version of openCV docs provides K1, K2, and K3, which is not supported by the Warp IP.

Rest all k0_pre, k0_post, k1_pre, k1_post, Q_fact_pre, Q_fact_post, k0_pre_Q_bits, k0_post_Q_bits, k1_pre_Q_bits, k1_post_Q_bits parameters are internally calculated and used by Warp Initializer IP.

The fields in the descriptor are explained in the following table:

Table 5: Warp Initializer Descriptor Fields

Field	Explanation
width	Width of the input source image. Supported values are between 320 and the value provided in the Maximum Number of Columns field in the Vivado IDE.
height	Height of the input source image. Supported values are between 240 and the value provided in the Maximum Number of Rows field in the Vivado Integrated Design Environment (IDE).
bytes_per_pixel	Number of bytes per pixel.
filter_table_addr_0	Memory address of warp filter table0.
filter_table_addr_1	Memory address of warp filter table1.
width_Q4	Width of the image in Q4 fixed point format.
height_Q4	Height of the image in Q4 fixed point format.
k_pre	Lens distortion value applied on the image before applying any other distortions. The user provided value is converted into a fixed format by the driver. Supported range without any other distortions is -5.0 to 5.0. The range will vary if other distortions are applied on images.
k_post	Lens distortion value applied on the image after applying all other distortions. The user provided value is converted into a fixed format by the driver. Supported range without any other distortions is -5.0 to 5.0. The range will vary if other distortions are applied on images.
k0_pre	Fixed point parameter estimated by the driver in onetime_calcs_warp() function. Its value depends on the k_pre.
k1_pre	Fixed point parameter estimated by the driver in onetime_calcs_warp() function. Its value depends on the k_pre.
k0_post	Fixed point parameter estimated by the driver in onetime_calcs_warp() function. Its value depends on the k_post.

Table 5: Warp Initializer Descriptor Fields (cont'd)

Field	Explanation
k1_post	Fixed point parameter estimated by the driver in onetime_calcs_warp() function. Its value depends on the k_post.
Q_fact_pre	Fixed point format used for k_pre. This parameter is estimated by the driver in onetime_calcs_warp() function.
Q_fact_post	Fixed point format used for k_post. This parameter is estimated by the driver in onetime_calcs_warp() function.
k0_pre_Q_bits	Fixed point format used for k0_pre. This parameter is estimated by the driver in onetime_calcs_warp() function.
k0_post_Q_bits	Fixed point format used for k0_post. This parameter is estimated by the driver in onetime_calcs_warp() function.
k1_pre_Q_bits	Fixed point format used for k1_pre. This parameter is estimated by the driver in onetime_calcs_warp() function.
k1_post_Q_bits	Fixed point format used for k1_post. This parameter is estimated by the driver in onetime_calcs_warp() function.
h[6]	Homography matrix estimated based on the inputs of the rotation, scale, transition and keystone distortion values. h[6] represents the first two columns of homography 3x3 matrix in fixed point format.
h_trans [3]	Represents the last column of homography 3x3 matrix in fixed point format.
h_Qbits[6]	Fixed point format used for each element of the h[6]. These parameters are estimated by the driver in onetime_calcs_warp() function.
normfactor	Fixed point value estimated by the driver in onetime_calcs_warp() function based on image width and height.
cenX	Center column position of the image estimated based on width.
cenY	Center row position of the image estimated based on height.
num_ctrl_pts	Number of control point used in arbitrary distortion. This parameter is required for arbitrary warp.
src_ctrl_x_pts	Memory address of the column locations of source control points. These parameters are input from the user for arbitrary warp.
src_ctrl_y_pts	Memory address of the row locations of source control points. These parameters are input from the user for arbitrary warp.
src_tangents_x	Memory address of the column direction tangents estimated by the driver in onetime_calcs_warp() function based on source and destination control points.
src_tangents_y	Memory address of the row direction tangents estimated by the driver in onetime_calcs_warp() function based on source and destination control points.
interm_x	Memory address for storing intermediate information during warp filter parameters estimation block by block.
interm_y	Memory address for storing intermediate information during warp filter parameters estimation block by block.
warp_type	Type of warp. 0 for regular and 1 for arbitrary distortion.
valid_segs	Number of valid blocks that the Warp Filter needs to reconstruct. This value must be read from the valid_segs field of the Warp Initializer descriptor.
lblk_count	For faster processing, the Warp Filter runs two cores in parallel. The total number of valid blocks is divided between the two cores. The number of blocks that need to be processed by the first core comes from lblock_count. This value must be read from the lblock_count field of the Warp Initializer descriptor.
line_num_seg	The line number from which the second core in the Warp Filter starts reading the source image. This value must be read from the line_num field of the Warp Initializer descriptor.

Note: If Valid Segments (`valid_segs`) are 0 for a particular configuration in Warp Initializer IP, the particular configuration is invalid and the warp filter interrupt is low. Further warp filter processing is not done.

This descriptor is filled in the driver based on the user input configuration. Provide the following input configuration structure to the driver.

```
typedef struct {
    int frame_width;
    int frame_height;
    int warp_type;
    float pre_fisheye;
    float post_fisheye;
    int left_top_x;
    int left_top_y;
    int right_top_x;
    int right_top_y;
    int left_bottom_x;
    int left_bottom_y;
    int right_bottom_x;
    int right_bottom_y;
    float scale_x;
    float scale_y;
    float rotation;
    float trans_x;
    float trans_y;
    float zoom;
    int num_ctrl_pts;
    XVWarpInit_ArbParam_MeshInfo *ctrl_pts;
} WARP_CFG;
```

The supported range for each parameter is given in the following table.

Table 6: Warp Configuration Parameters

Warp Configuration Parameter	Range
frame_width	320 to 3840
frame_height	240 to 2160
warp_type	0 (fixed), 1 (arbitrary)
pre_fisheye	-5.0 to 5.0
post_fisheye	-5.0 to 5.0
left_top_x	0 to width
left_top_y	0 to height
right_top_x	0 to width
right_top_y	0 to height
left_bottom_x	0 to width
left_bottom_y	0 to height
right_bottom_x	0 to width
right_bottom_y	0 to height
scale_x	0.5 to 2.0
scale_y	0.5 to 2.0

Table 6: Warp Configuration Parameters (cont'd)

Warp Configuration Parameter	Range
rotation	-90 to 90
trans_x	-width to width
trans_y	-height to height
zoom	0.5 to 2
num_ctrl_pts	4, 8, 16, 32

The following figures show the effect of these values on a 3840x2160 image.

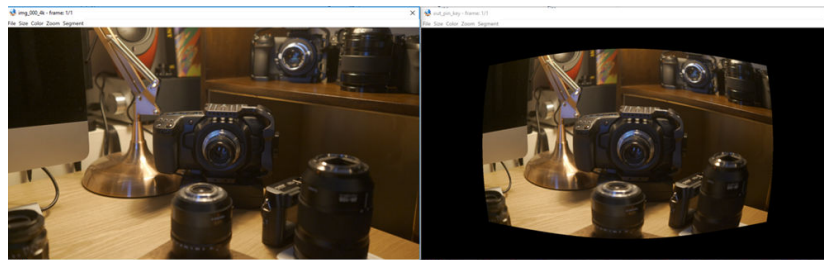
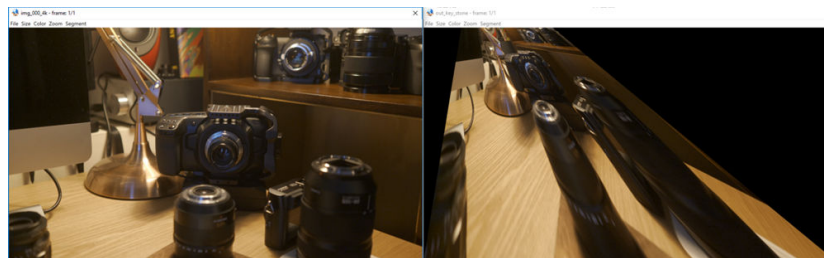
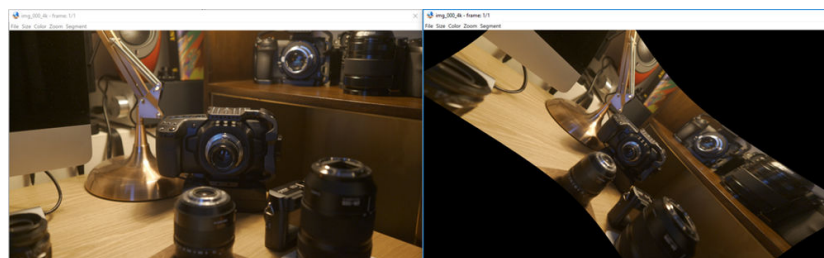
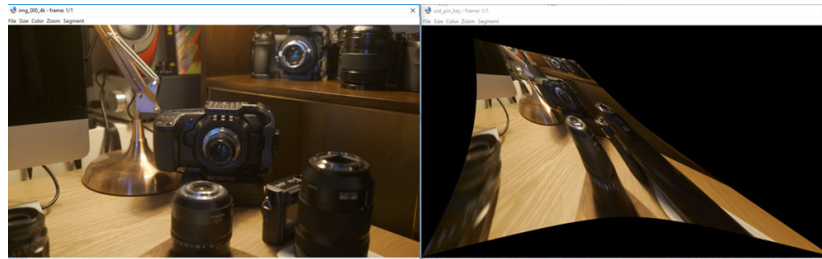
Figure 4: **post_fisheye = -2.0, scale_x = 0.75, scale_y = 0.75**Figure 5: **left_top_x = 400, left_top_y = 0, right_top_x = 1600, right_top_y = 0, left_bottom_x = 0, left_bottom_y = 2160, right_bottom_x = 3840, right_bottom_y = 2160**Figure 6: **post_fisheye = 2.0, rotation = 45**

Figure 7: `post_fisheye = 2.0`, `left_top_x = 400`, `left_top_y = 0`, `right_top_x = 1600`, `right_top_y = 0`, `left_bottom_x = 0`, `left_bottom_y = 2160`, `right_bottom_x = 3840`, `right_bottom_y = 2160`



Register Address Space for Warp Filter

Table 7: Register Address Space for Warp Filter

Address (hex) BASEADDR+	Register Name	Access Type	Register Description
0x0000	Control Signals	R/W	Bit[0] = <code>ap_start(R/W/COH)</code> ²
			Bit[1] = <code>ap_done(R/COR)</code> ³
			Bit[2] = <code>ap_idle(R)</code>
			Bit[3] = <code>ap_ready(R)</code>
			Bit[7] = <code>auto_restart(R/W)</code>
			Others = Reserved
0x00004	Global Interrupt Enable	R/W	Bit[0] = Global interrupt enable
			Others = Reserved
0x00008	IP Interrupt Enable	R/W	Bit[0] = <code>ap_done</code>
			Bit[1] = <code>ap_ready</code>
			Others = reserved
0x0000C	IP Interrupt Status register	R/TOW ¹	Bit[0] = <code>ap_done</code>
			Bit[1] = <code>ap_ready</code>
			Others = Reserved
0x00010	Descriptor Address register 0	R/W	Bit[31] to Bit[0] = LSB 32 bits of the descriptor address
0x00014	Descriptor Address register 1	R/W	Bit[31] to Bit[0] = MSB 32 bits of the descriptor address

Notes:

1. TOW – toggle on write.
2. COH – clear on handshake.
3. COR – clear on read.
4. The Control register (0x0000), Global Interrupt Enable register (0x0004), IP Interrupt Enable register (0x0008), and IP Interrupt Status register (0x000C) are explained in section [S_AXILITE Control Register Map](#) of UG1399. In UG1399, these register definitions might have some additional bits, but in this IP, only bits mentioned in this table are accessed. Consider only these bits when accessing the Control register, Global Interrupt Enable register, IP Interrupt Enable register, and IP Interrupt Status register.

Register Description

- **Control (0x00000) register:** This register controls the operation of the Filter. Bit[0] of the Control register, `ap_start`, kicks off the core from software. Writing 1 to this bit starts the core to generate a video frame. To set the core in free running mode, Bit[1] of the Control register, `ap_done`, indicates when the IP has completed all operations in the current transaction. A logic 1 on this signal indicates that the IP has completed all operations in this transaction.

Bit[2] of the Control register, `ap_idle`, signal indicates if the IP is operating or idle (no operation). The idle state is indicated by logic 1. This signal is asserted Low when the IP starts operating. This signal is asserted High when the IP completes operation and no further operations are performed.

Bit[3] of the Control register, `ap_ready`, signal indicates when the IP is ready for new inputs. It is set to logic 1 when the IP is ready to accept new inputs, indicating that all input reads for this transaction are completed. If the IP has no operations in the pipeline, new reads are not performed until the next transaction starts. This signal is used to make a decision on when to apply new values to the input ports and whether to start a new transaction. Bit[3] to Bit[6] are not used.

Bit[7] of the Control register, `auto_restart`, can be set to enable the auto-restart mode and then the IP restarts automatically at the end of each transaction.

- **Global Interrupt Enable (0x00004) register:** This register is the master control for all interrupts. Bit[0] can be used to enable or disable all core interrupts.
- **IP Interrupt Enable (0x00008) register:** This register allows interrupts to be enabled selectively. Currently, two interrupt sources are available: `ap_done` and `ap_ready`. `ap_done` is triggered after the frame processing is complete, and `ap_ready` is triggered after the core is ready to start processing the next frame.
- **IP Interrupt Status (0x0000C) register:** This is a dual-purpose register. When an interrupt occurs, the corresponding interrupt source bit is set in this register. In readback mode (get status), the interrupting source can be determined. In writeback mode (clear interrupt), the requested interrupt source bit is cleared.
- **IP Descriptor Address Register 0 (0x00010) register:** This register allows to configure the LSB 32-bit of the 64-bit descriptor address to the IP.
- **IP Descriptor Address Register 0 (0x00014) register:** This register allows to configure the MSB 32-bit of the 64-bit descriptor address to the IP.

Warp Filter Descriptor Structure

The Warp Filter descriptor should be placed in DDR memory in the following format. The address of this descriptor is provided to the IP through the Descriptor Address register and the IP reads the descriptor from this address.

```
typedef struct {
    u32 height;
    u32 width;
    u32 stride;
    u32 format;
    u32 valid_seg;
    u32 lblock_count;
    u32 line_num;
    u32 reserved;
    u64 src_buf_addr;
    #if ((MAX_NR_PLANES == 2) || (MAX_NR_PLANES == 3))
    u64 src_buf_addr1;
    #endif
    #if (MAX_NR_PLANES == 3)
    u64 src_buf_addr2;
    #endif
    u64 seg_table_addr;
    u64 dest_buf_addr;
    #if ((MAX_NR_PLANES == 2) || (MAX_NR_PLANES == 3))
    u64 dest_buf_addr1;
    #endif
    #if (MAX_NR_PLANES == 3)
    u64 dest_buf_addr2;
    #endif
    #if (WRITE_INVALID == 1)
    AXIMM_WRITE dest_buf_addr_i;
    AXIMM_READ seg_table_addr_i;
    #endif
    u64 Warp_NextDescAddr;
} XWVWarpFilter_Desc;
```

The fields in the above descriptor are explained as below:

Table 8: Warp Filter Descriptor Fields

Descriptor Field	Explanation
height	Height of the input source image. Supported values are between 240 and the value provided in the Maximum Number of Rows field in the Vivado IDE.
width	Width of the input source image. Supported values are between 320 and the value provided in the Maximum Number of Columns field in the Vivado IDE.
stride	Stride in bytes of the source image.
format	Pixel format of the source image.
valid_seg	Number of valid blocks that the Warp Filter needs to reconstruct. This value must be read from the valid_segs register of the Warp Initializer.
lblock_count	For faster processing, the Warp Filter runs two cores in parallel. The total number of valid blocks are divided between the two cores. The number of blocks that need to be processed by the first core comes from lblock_count. This value must be read from the lblock_count register of the Warp Initializer IP.

Table 8: Warp Filter Descriptor Fields (cont'd)

Descriptor Field	Explanation
line_num	The line number from which the 2 nd core in the Warp Filter starts reading the source image. This value must be read from the line_num register of the Warp Initializer.
src_buf_addr	The 64-bit address from which the source image must be read.
src_buf_addr1	The 64-bit address from which the source image must be read.
src_buf_addr2	The 64-bit address from which the source image must be read.
seg_table_addr	The 64-bit address from which the Warp Filter parameters must be read.
dest_buf_addr	The 64-bit address to which the corrected image must be written.
dest_buf_addr1	The 64-bit address to which the corrected image must be written.
dest_buf_addr2	The 64-bit address to which the corrected image must be written.

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

Clocking

The Warp Initializer and Warp Filter cores have only one clock domain. All the interfaces, (the AXI4 interface), and the memory mapped AXI4 interface use the `ap_clk` pin as their clock source.

System Considerations

The Warp Filter must be configured correctly to operate properly. There should be sufficient memory bandwidth available for this core to function properly. The bandwidth required (in MB/s) for a memory layer can be calculated with the following equation:

- S (bandwidth required for source image read in MB/s) = $FPS \times height \times stride$
- D (bandwidth required for destination image write in MB/s) = $FPS \times height \times stride$
- R (bandwidth required for filter parameters read in MB/s) = $FPS \times height \times width \times 4$
- Total bandwidth required in MB/s = $S + D + R$

Where *FPS* is the number of frames per second the IP is operating at, *height* is the height in lines of the image, and *stride* is the stride in bytes of the image.

Note: The preceding number is an approximate bandwidth requirement. The actual bandwidth requirement varies with the distortion parameters.

For example, for a 3840x2160 @ 60 RGB 12-bit video, the bandwidth requirement is calculated as follows:

- $S = 60 \times 2160 \times 3840 \times 5$
- $D = 60 \times 2160 \times 3840 \times 5$
- $R = 60 \times 2160 \times 3840 \times 4$

- Total bandwidth required = ~ 7 GB/s

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the `Customize IP` command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

Configuration Tab

The Warp Processor is configured to meet your specific requirements through the AMD Vivado™ Design Suite. This section provides a quick reference to parameters that can be configured at generation time.

The following figures show the main configuration screens of the Warp Initializer and Warp Filter IP in the Vivado Design Suite.

Note: For both IPs, the configuration must be the same.

Figure 8: Warp Initializer Configuration Screen in the Vivado Design Suite

warp initializer (1.1)

Documentation IP Location

☐ Show disabled ports

Component Name: v_warp_init_0

Maximum Number of Columns: 4096 [320 - 4096]

Maximum Number of Rows: 2160 [240 - 2160]

Number of regions: 4

Bits per component: 16

Bits per component: 8

Number of BRAM lines: 64

AXIMM Read data width: 128

AXIMM write data width: 128

remap vector size: 32

WARP TYPE

☒ FIXED

☐ KEYSTONE

☐ ARBITRARY

☐ BOTH

Maximum CTRL PTS: 32

INTERPOLATION

☐ BILINEAR

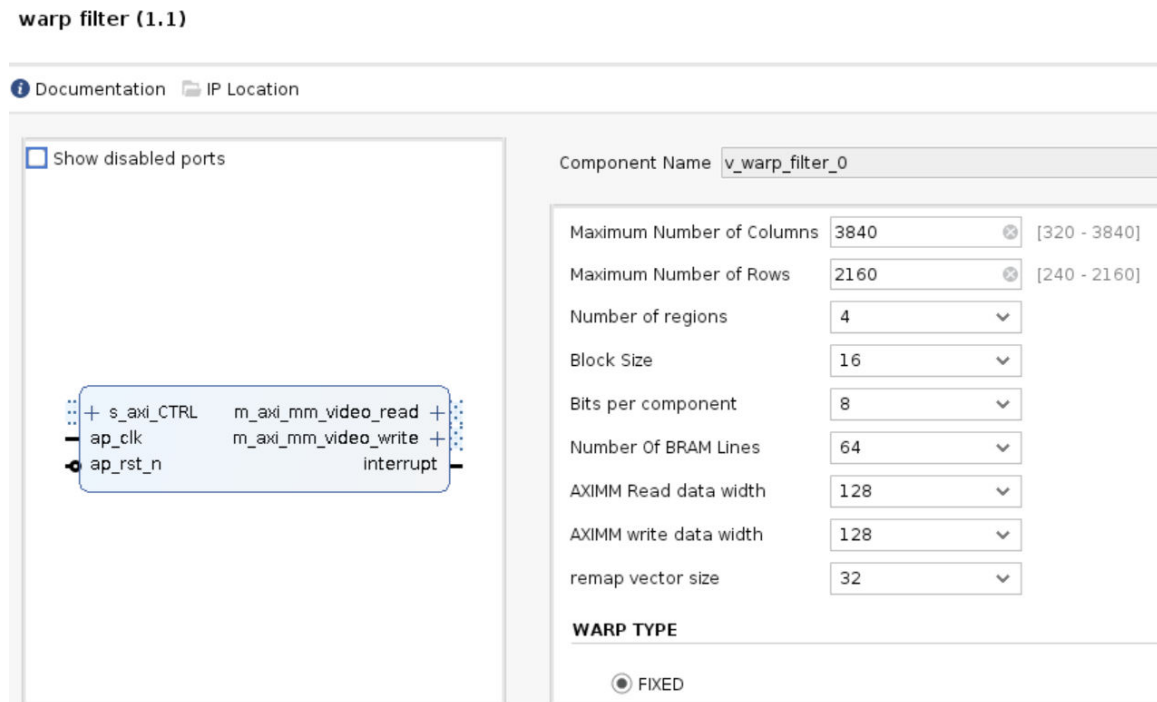
☒ BICUBIC

Performance Level: 0

Use UltraRAM: 0

Ports: s_axi_CTRL, ap_clk, ap_rst_n, m_axi_mm_video, interrupt

Figure 9: Warp Filter Configuration Screen in the Vivado Design Suite



The following settings are generally applicable to both the Warp Initializer and the Warp Filter IP.

- **Component Name:** The component name is used as the base name of output files generated for the module. Names must begin with a letter and must be composed from characters: a to z, 0 to 9 and "_".
- **Maximum Number of Columns:** Specifies the maximum number of active video columns/pixels the IP core can produce at runtime. Any video width that is less than the Maximum Number of Columns can be programmed through the AXI4-Lite control interface without regenerating the core.
- **Maximum Number of Rows:** Specifies the maximum number of active video rows/lines the IP core can produce at runtime. Any video height that is less than the Maximum Number of Rows can be programmed through the AXI4-Lite control interface without regenerating the core.
- **Number of Regions:** Specifies the number of regions into which the source image is divided vertically. The Warp Filter IP reads lines from each region one after the other. Permitted values are 1, 2, and 4.
- **Segment Width:** Specifies the width of the output block. Permitted values are 8, 16, and 32.
- **Segment Height:** Specifies the height of the output block. Permitted values are 8, 16, and 32.
- **Bits per component:** Specifies the bit depth of the input pixels. Permitted values are 8, 10, and 12.

- **Number of Block RAM lines:** Specifies the number of lines the cache in the Warp Filter IP can hold.
- **AXI MM Read Data Width:** Specifies the data width of the AXI MM interface used for reading the source image and Warp Filter parameters.
- **AXI MM Write Data Width:** Specifies the data width of the AXI MM interface used for writing the output.
- **Warp Type:** Specifies the type of distortion.
 - **FIXED:** The IP supports fixed distortion only (pincushion, barrel, projective, rotation, translation, and scaling).
 - **ARBITRARY:** The IP supports arbitrary distortion only.
 - **BOTH:** The IP supports both fixed and arbitrary distortion types.
- **Interpolation:** Specifies the warp filter quality. It can be high quality or moderate quality.
 - **BILINEAR:** Moderate quality.
 - **BICUBIC:** High quality.
- **Performance Level:** Specifies the performance level of the IP.
 - **0:** Can support up to 4 k @ 30 FPS
 - **1:** Can support up to 4 k @ 60 FPS

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Constraining the Core

Required Constraints

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

The simulation of this core is not supported.

Synthesis and Implementation

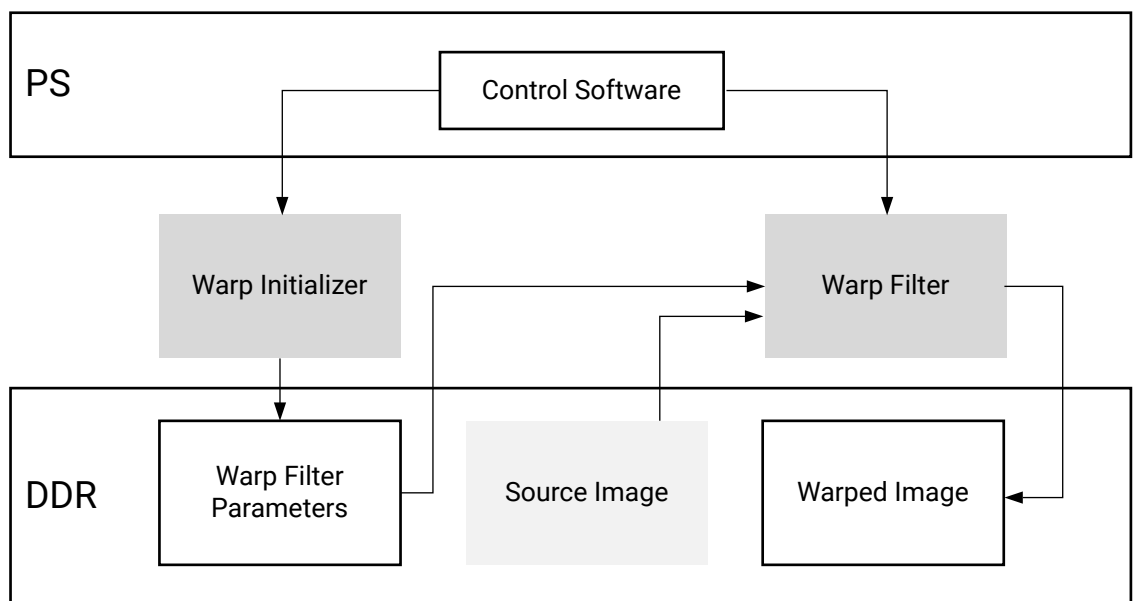
For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Example Design

Warp Processor is a memory based IP. In this example design, the Warp Initializer generates the Warp Filter parameters for a distortion selected by the user and writes it to the memory. After getting the interrupt from the Warp Initializer, the Warp Filter reads the source image and Warp Filter parameters from the memory, and writes the distorted image back to memory at the destination address. The Warp Filter IP then generates a frame done interrupt that is connected to the AMD Zynq™ UltraScale+™ MPSoC.

The application code programs the distortion parameters that you provide into the hardware registers of the Warp Initializer and starts the IP. The Warp Initializer writes the Warp Filter parameters to the memory and generates an interrupt. On receiving the interrupt, the application code programs the input video parameters to the Warp Filter and starts the IP. The Warp Filter IP reads the source image data and Warp Filter parameters, performs interpolation, and writes the output to the destination buffer.

Figure 10: Warp Processor Block Diagram



X25537-070921

This chapter provides an example system that includes the Warp Initializer and Warp Filter IPs. Important system-level aspects when designing with the Warp Processor IPs are highlighted in these example designs, including the following:

- Typical usage of the Warp Initializer and the Warp Filter IPs in conjunction with other cores
- Usage with memory mapped AXI4 interface memory buffers
- Configuration of both the IPs by programming the registers.

The supported platforms are listed in the following table.

Table 9: Supported Platforms

Development Boards	Additional Hardware	Processor
ZCU102	N/A	psu_cortexa53_0

To open the example project, perform the following steps:

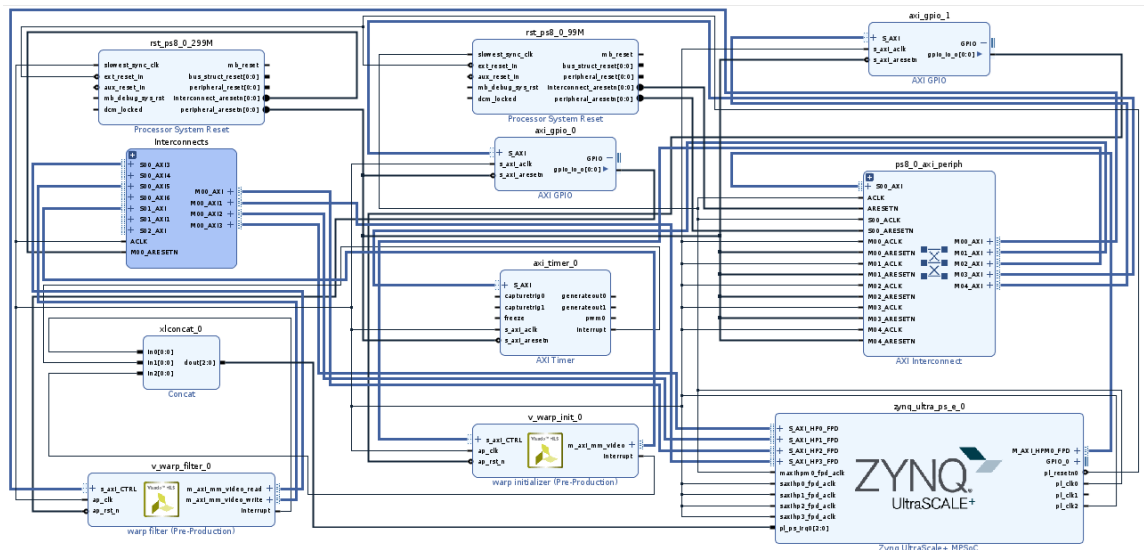
1. Select the Warp Initializer IP and Warp Filter IP from the Vivado IP catalog.
2. Double-click the selected IP or right-click the IP and select **Customize IP** from the menu.
3. Configure the build-time parameters in the **Customize IP** window and click **OK**. The Vivado IDE generates an example design matching the build-time configuration.
4. In the Generate Output Products window, select **Generate** or **Skip**. If Generate is selected, the IP output products are generated after a brief moment.
5. Right-click on either the Warp Initializer IP or Warp Filter IP in the Sources panel and select **Open IP Example Design** from the menu.
6. In the Open IP Example Design window, select **example project directory**, and click **OK**.

The Vivado software then runs automation to generate the example design in the selected directory. The generated project contains a synthesizable example design. The synthesizable example block design, along with the top-level file, resides in the Design Sources catalog.

Synthesizable Example Design

The synthesizable design uses the Zynq UltraScale+ MPSoC microprocessor as AXI4 master. The interrupt port of the Warp Initializer IP and the Warp Filter IP is connected to the Zynq UltraScale+ MPSoC. The Warp Filter IP sends the interrupt after generating the corrected output.

Figure 11: Synthesizable Example Design



The synthesizable example design requires both AMD Vivado™ and AMD Vitis™ tools.

The first step is to run synthesis, implementation, and bitstream generation in Vivado. After all those steps are done, select **File → Export → Export Hardware**.

In the window, select **Include** bitstream, select an export directory, and click **OK**.

The remaining work is performed in the Vitis tool. The Warp Processor example design file can be found in the following Vitis directory: `/data/embeddedsw/XilinxProcessorIPLib/drivers/v_warp_filter/examples/`.

The example application design source files (contained within the `examples` folder) are tightly coupled with the example design available in the Vivado IP catalog.

Later, perform the following steps to run the software application:



IMPORTANT! Make sure that the hardware is powered on and a Digilent Cable or a USB Platform Cable is connected to the host PC. Also, ensure that a USB cable is connected to the UART port of the ZCU102 board.

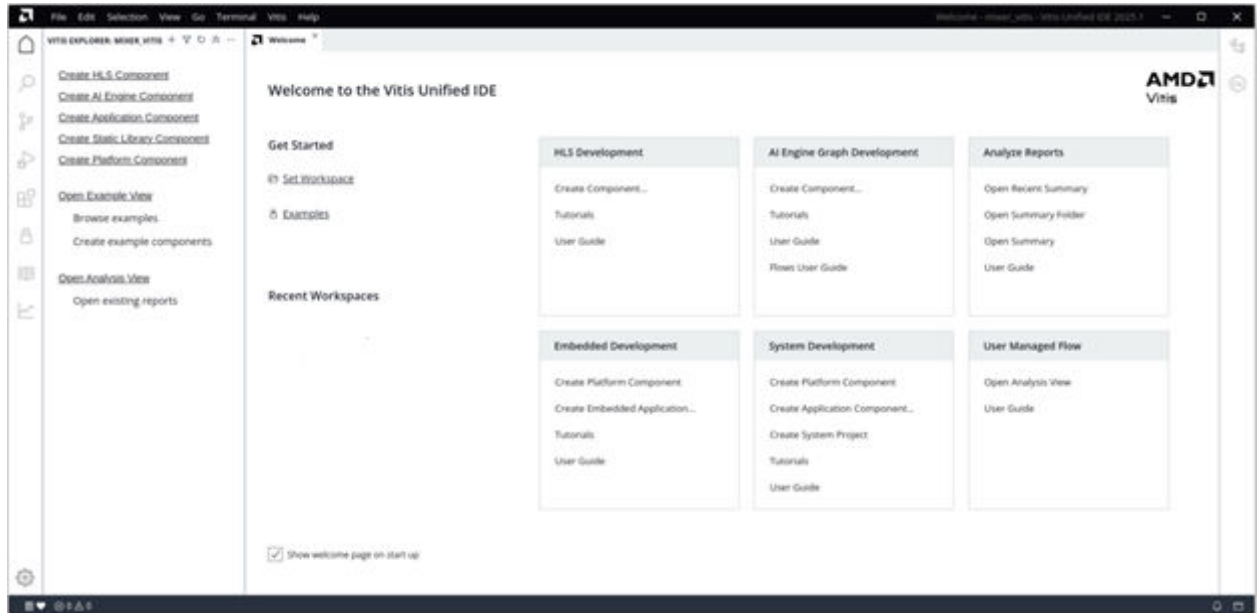
The example design test results are shown in the terminal program.

When executed on the board, the operations are listed in `readme.txt` in the `examples` folder. The video inputs tested are 1080p and 720p.

Obtaining the .elf File

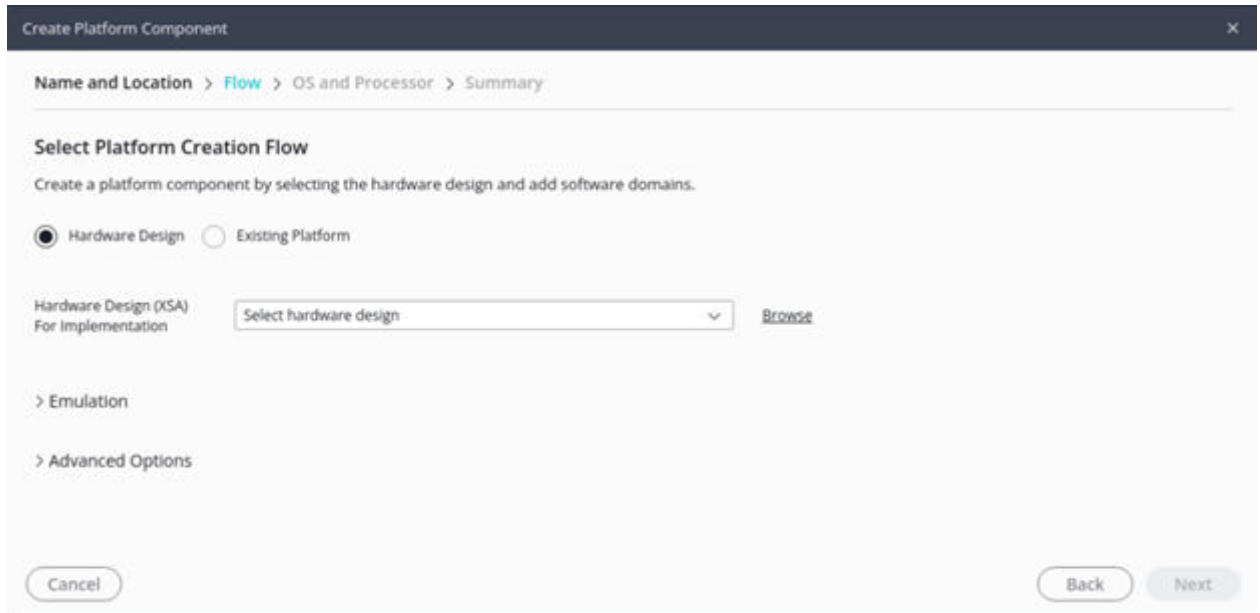
Perform the following steps to get the .elf file from the AMD Vitis™ software platform.

1. Open the Vitis application, and select **Get Started → Set Workspace**.



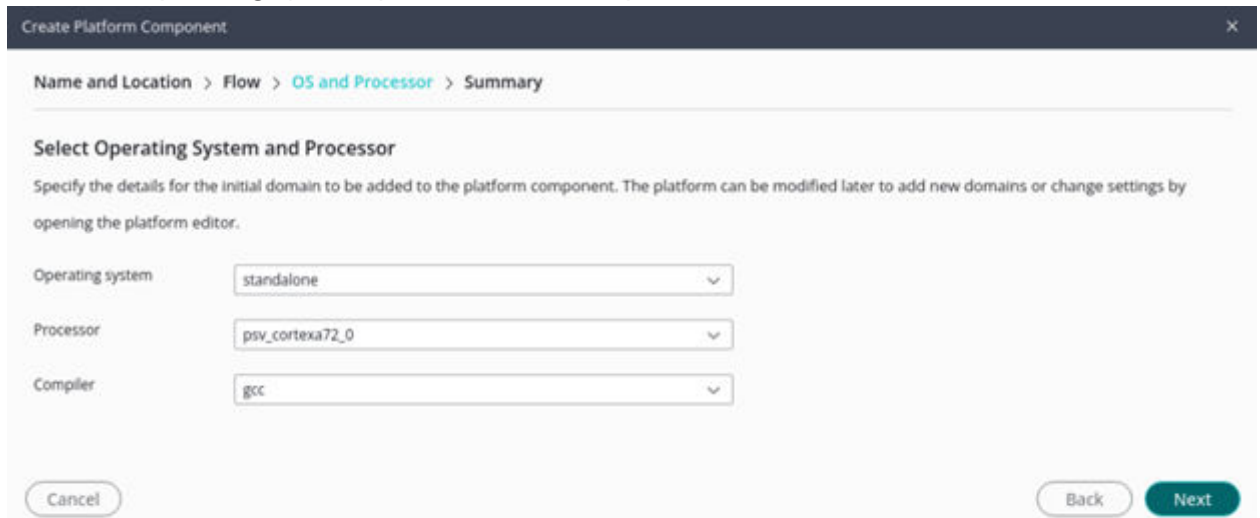
2. Create the platform component. Select **Embedded Development** → **Create Platform Component**.
3. Enter the component name and location, and click **Next**.

4. Browse and select the hardware XSA file generated with Vivado.



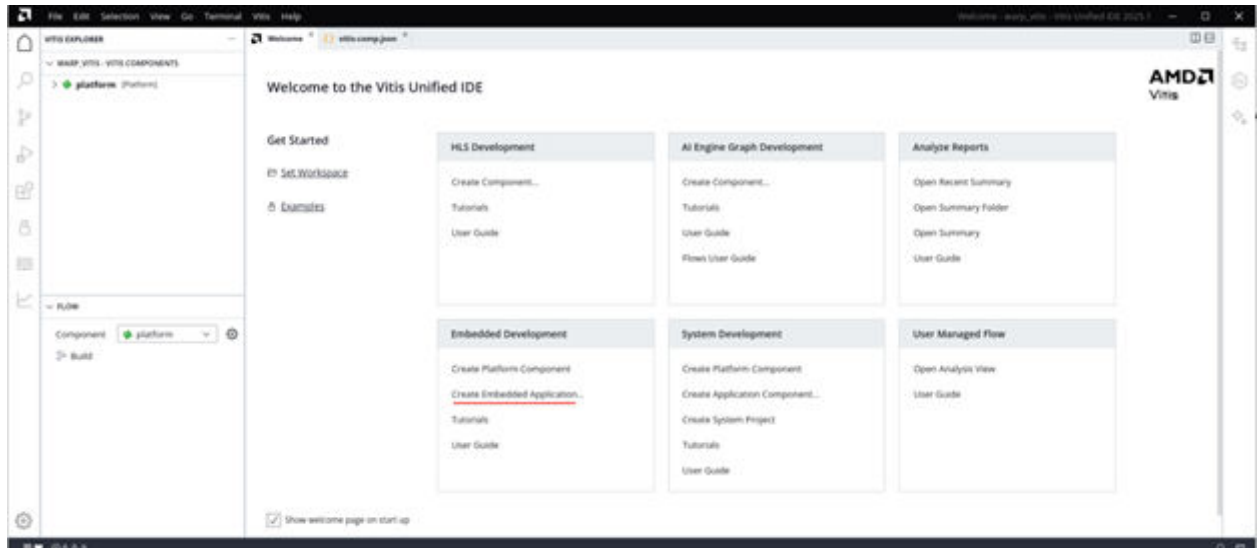
The screenshot shows the 'Create Platform Component' dialog with the 'Flow' step selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Platform Creation Flow' instructs the user to 'Create a platform component by selecting the hardware design and add software domains.' There are two radio buttons: 'Hardware Design' (selected) and 'Existing Platform'. Below, the 'Hardware Design (XSA) For Implementation' section has a dropdown menu showing 'Select hardware design' and a 'Browse' button. At the bottom, there are 'Emulation' and 'Advanced Options' expandable sections, and 'Cancel', 'Back', and 'Next' buttons.

5. Select the operating system, processor, and compiler.

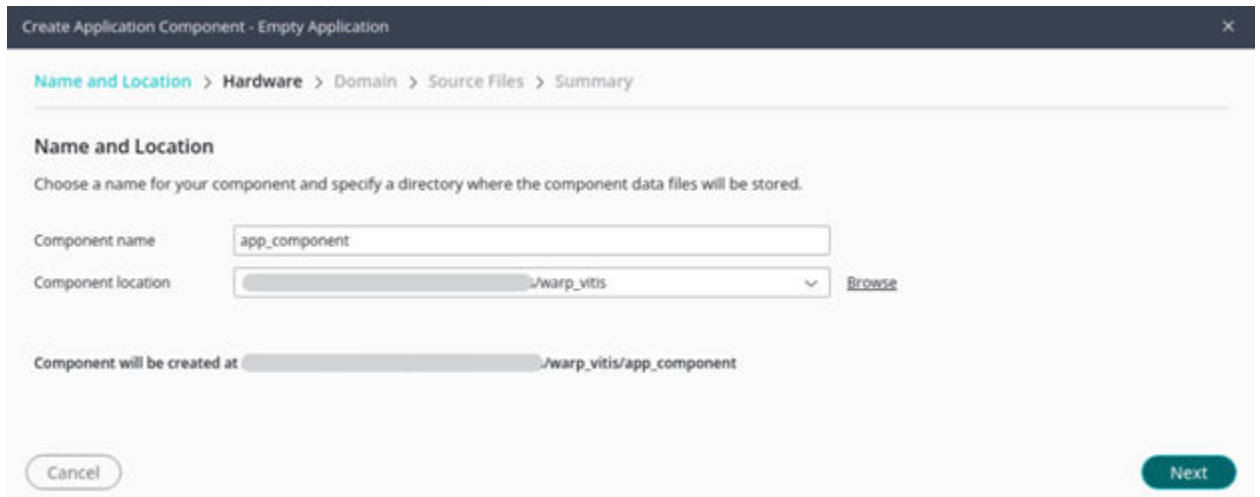


The screenshot shows the 'Create Platform Component' dialog with the 'OS and Processor' step selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Operating System and Processor' instructs the user to 'Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.' There are three dropdown menus: 'Operating system' (set to 'standalone'), 'Processor' (set to 'psv_cortexa72_0'), and 'Compiler' (set to 'gcc'). At the bottom, there are 'Cancel', 'Back', and 'Next' buttons.

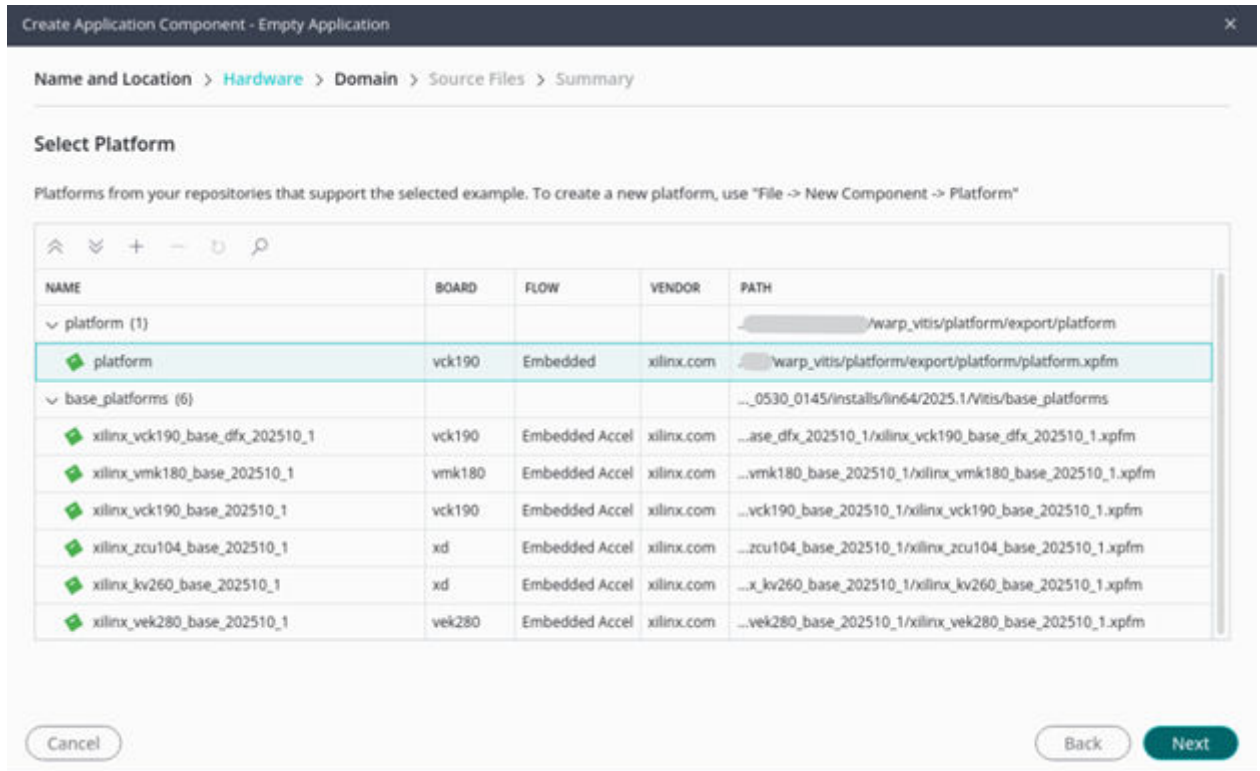
6. Review the summary, and click **Finish**.
7. After all the platform component is created, navigate back to the welcome screen, and under **Embedded Development** → **Create Embedded Application**, select **Create Embedded Application**.



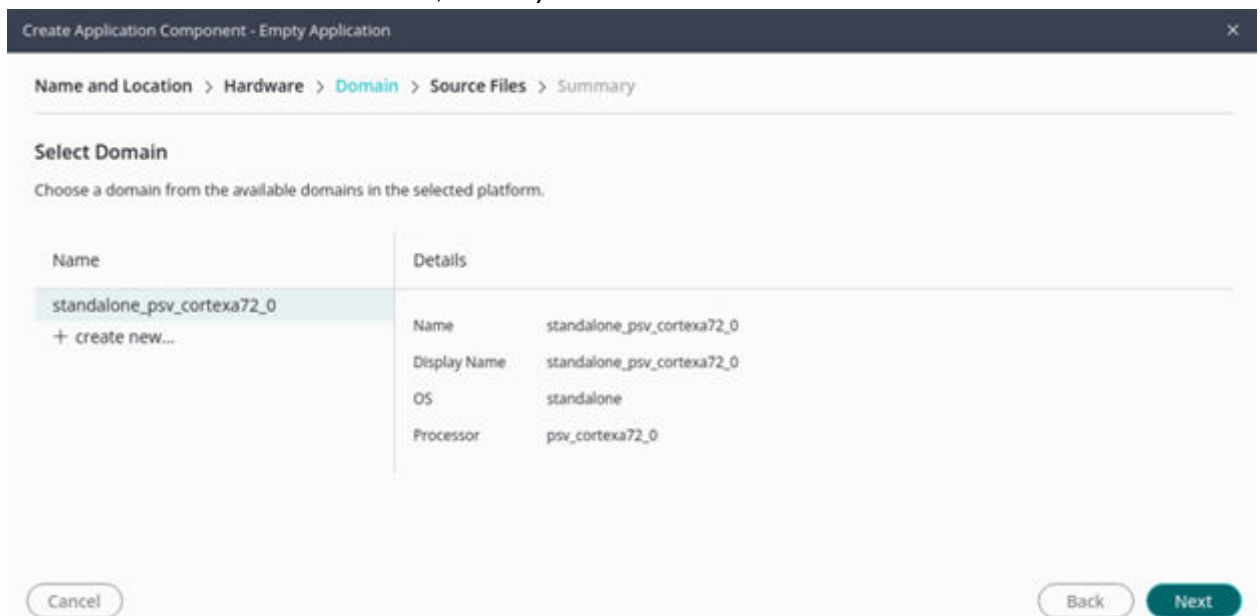
8. Select the component name and component location, and click **Next**.



9. In the Hardware tab, select the required platform, and click **Next**.

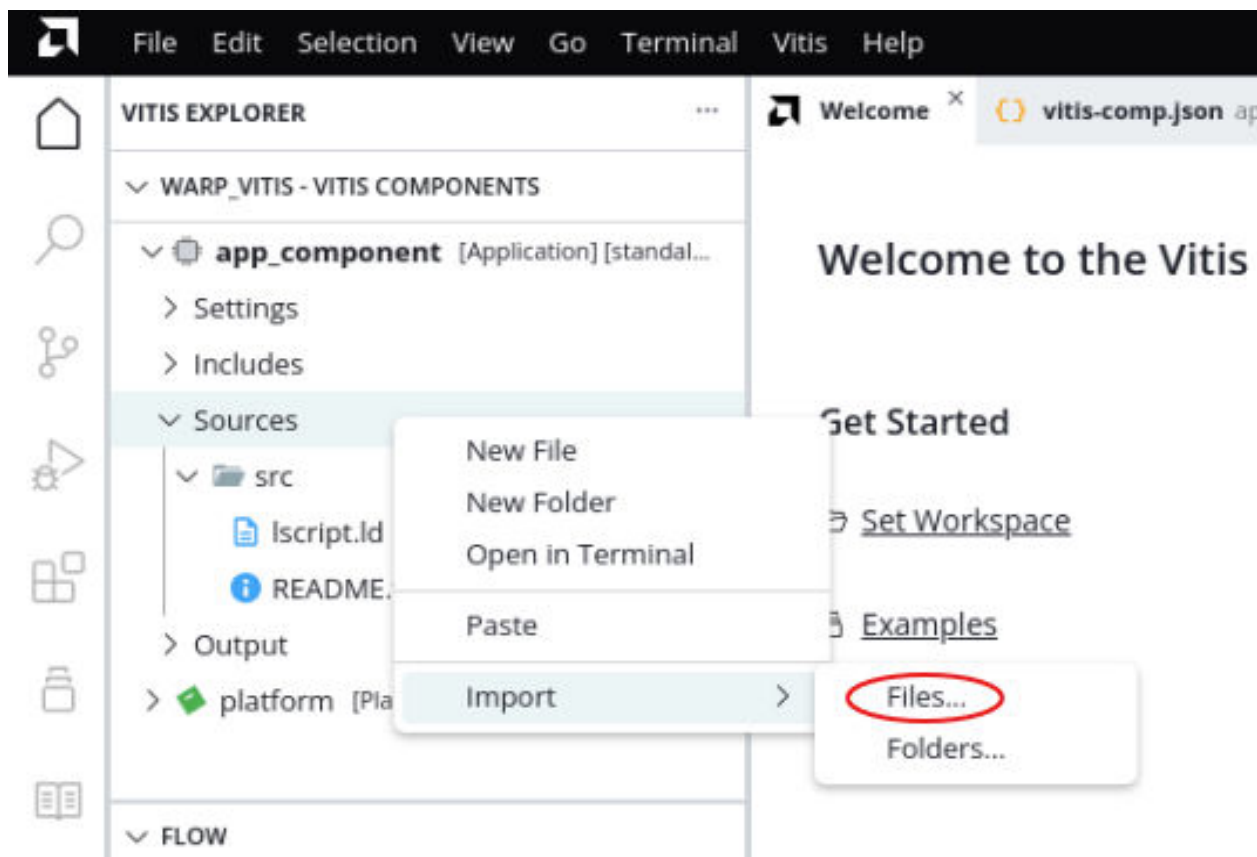


10. From the list of available domains, select your domain.

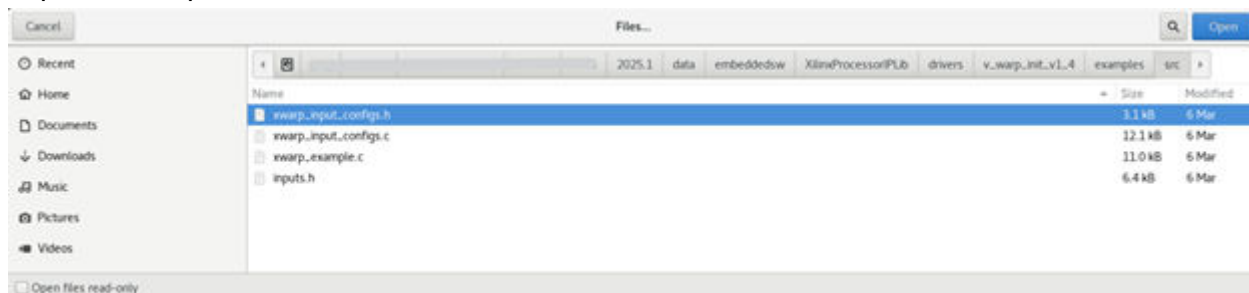


11. (Optional) Add the source files, review the summary, and click **Finish**.

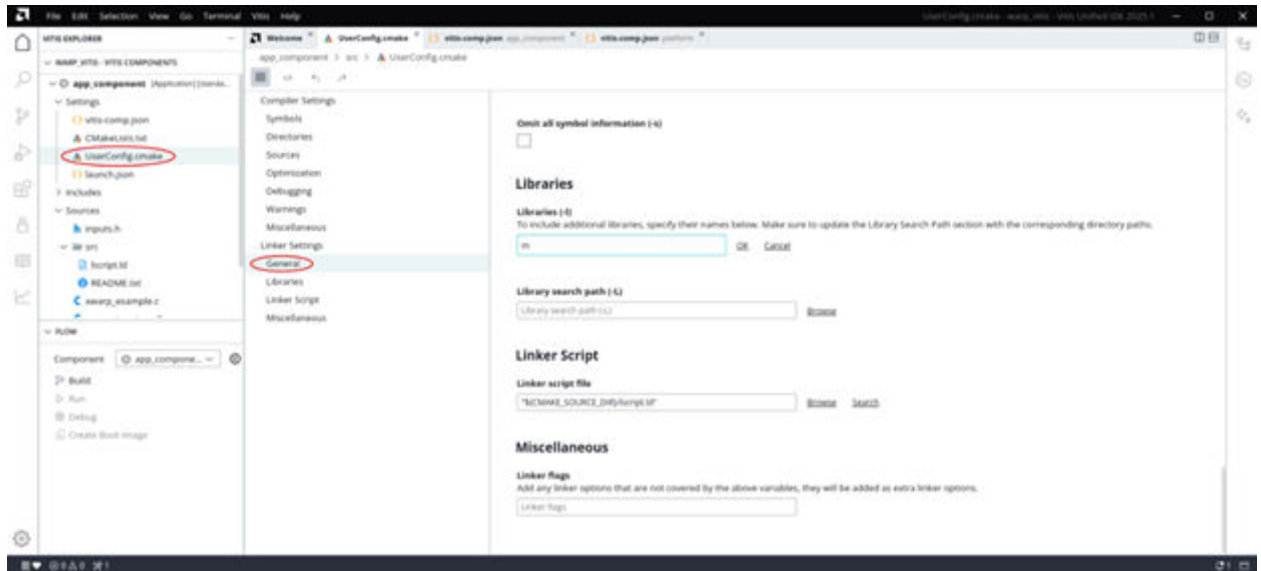
12. If skipped in the previous step, to import the source files into the app component, right-click **Sources**, and select **Import** → **Files**.



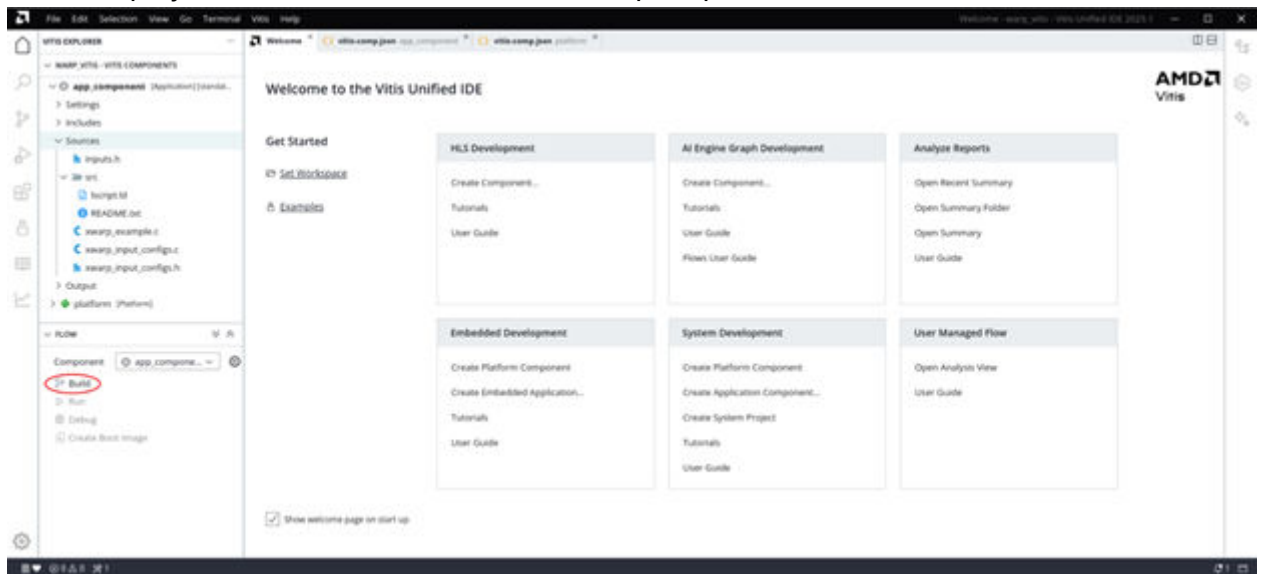
13. Import the required files.



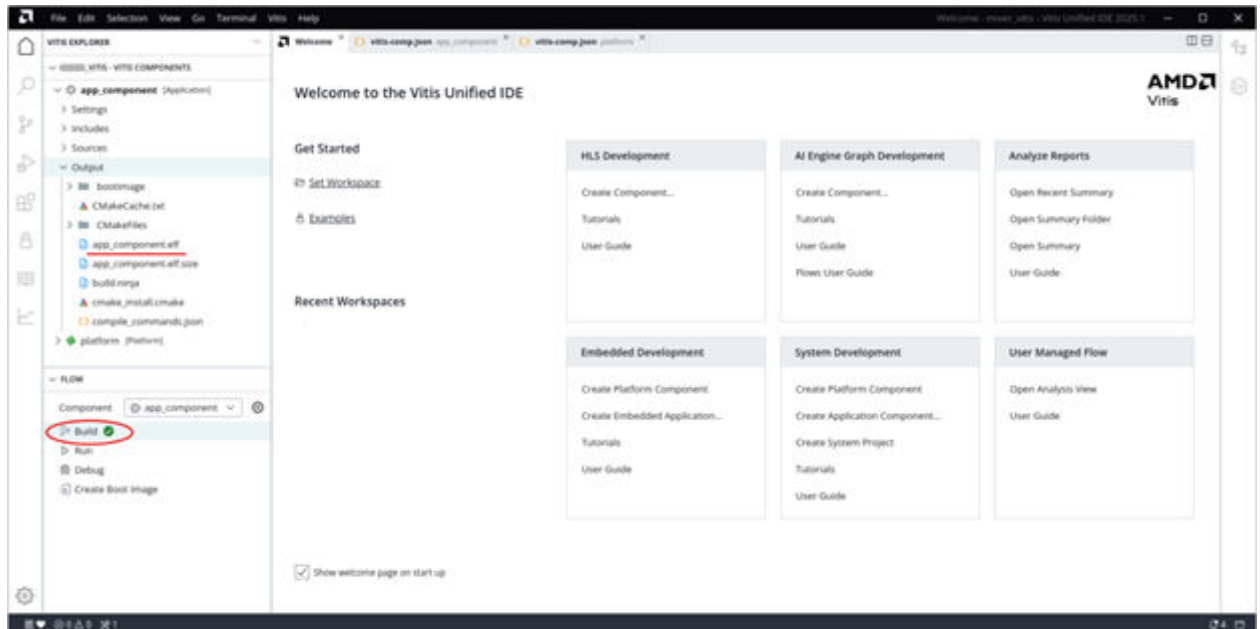
The Math Library is required to build the .elf file successfully.



14. Build the project. Select **Flow**→**Build**, and when prompted for Platform build, click **Yes**.



15. After the build is complete, check the Output folder for the .elf file.



16. Download the bitstream into the FPGA. Select **Vitis** → **Program Device**..
17. In the Program Device dialog box, ensure that the Bitstream field displays the bitstream file generated by the Tcl script, and click **Program**.
Note: If the programming is successful, the DONE LED on the board turns green.
18. A terminal program (HyperTerminal or PuTTY) is required for UART communication. Open the program, choose the appropriate port, set the baud rate to 115200, and establish a serial port connection.
19. In the Explorer panel, select and right-click **v_warp_example_design**.
20. Select **Run As** → **Launch on Hardware (GDB)**.
21. Select **Binaries and Qualifier**, and click **OK**.

Verification, Compliance, and Interoperability

This appendix provides details about how this IP core was tested for compliance with the protocol to which it was designed.

Simulation

A highly parameterizable test bench was used to test both Warp Initializer IP and Warp Filter IP in the AMD Vitis™ High- Level Synthesis (HLS) tool. Testing included the following:

1. Register accesses
2. Processing multiple frames of data
3. Varying IP throughput and pixel data width
4. Testing of various frame sizes
5. Varying parameter settings
6. Generating multiple outputs

Hardware Testing

The Warp Processor IPs have been validated at AMD to represent many different parameterizations. A test design was developed for the core that incorporated a Zynq UltraScale+ MPSoC processor, AXI4-Lite interconnect, and various other peripherals. The Zynq UltraScale+ MPSoC processor is responsible for the following functions:

1. Programming the IP registers.
2. Launching the test.
3. Reporting the pass/fail status of the test and any errors that were found.

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the [Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Community Forums](#) are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [AMD Adaptive Support web page](#) or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [AMD Adaptive Support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR [76221](#)

Technical Support

AMD Adaptive Computing provides technical support on the [Community Forums](#) for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Community Forums](#).

Debug Tools

There are many tools available to address Video Warp Processor design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The AMD Vivado™ debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.
- If your outputs go to 0, check your licensing.

Application Software Development

The Warp Initializer and Warp Filter cores are delivered with bare-metal drivers as part of the AMD Vitis™ software platform installation. The driver follows a layered architecture, where layer 1 provides basic register peek/poke capabilities and requires you to be familiar with the register map and inner workings of the core. Layer 2 abstracts away all the lower level details and provides an easy to use functional interface to the IP. AMD recommends using layer 2 APIs to interact with the core.

Building the BSP

While building the board support package (BSP), the driver extracts the Warp Initializer and Warp Filter IP hardware configuration settings from the provided hardware design file.

Modes of Operation

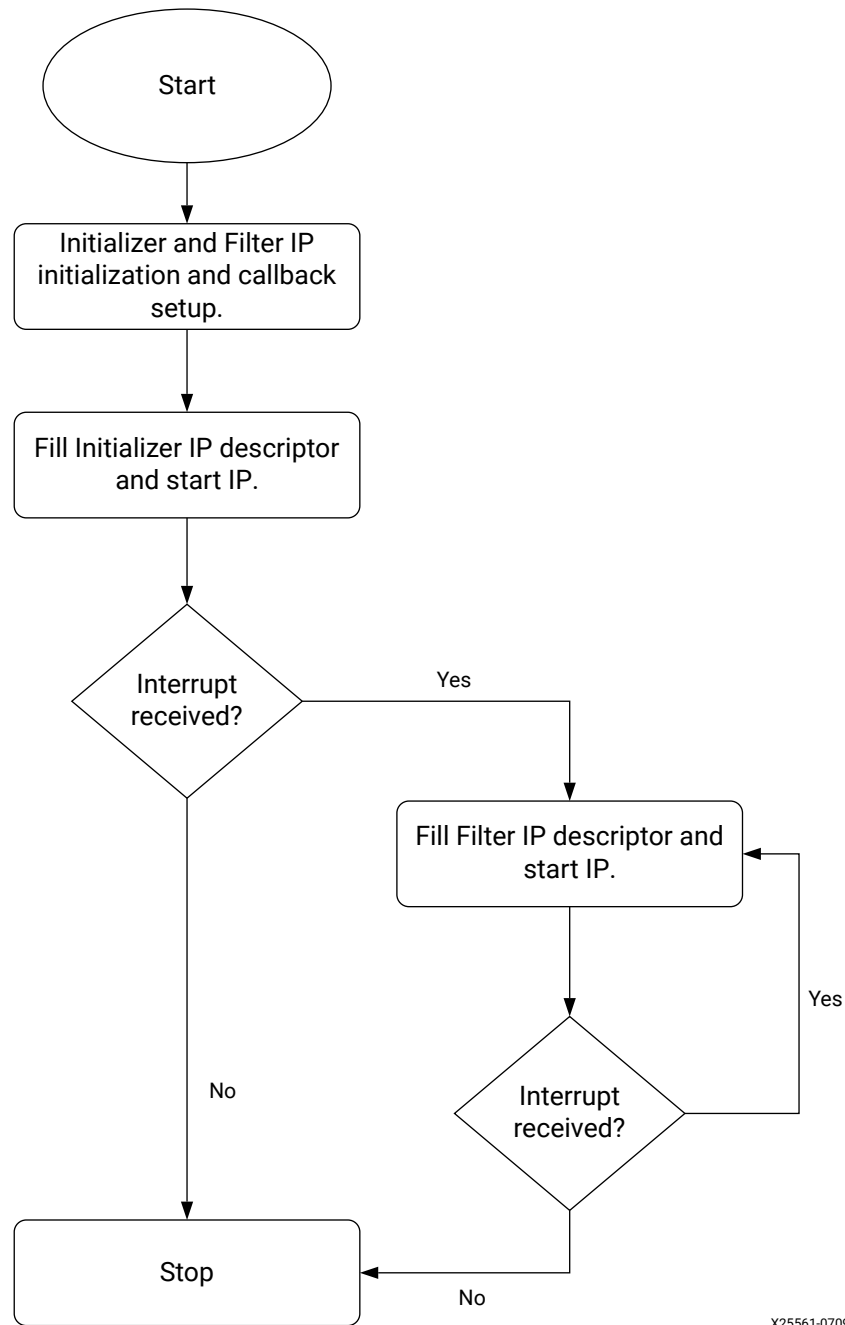
Both IPs support the interrupt mode of operation. In interrupt mode, the interrupt (IRQ) port of the core needs to be connected to a system interrupt controller. When an interrupt is triggered, the core interrupt service routine (ISR) checks to confirm if the current frame processing is complete. It then calls a user programmable callback function, if any. In the callback function, the register settings for the next frame should be programmed, that is, which source memory buffer address a frame should read from next. Finally, the interrupt service routine triggers the core to start processing the next frame. An application must perform the following tasks to configure the core for the interrupt mode:

- Register the Warp Initializer core ISR routine `XVWarpInit_IntrHandler` and Warp Filter core ISR routine `XVWarpFilter_IntrHandler` with the system interrupt controller.
- Register the application callback function that should be called within the interrupt context. This can be done using the API `XWarpInit_callback` for the Warp Initializer interrupt, and `XWarpFilter_callback` for the Warp Filter IP.
- Enable the interrupts for both IPs by calling API `XVWarpInit_EnableInterrupts` and `XVWarpFilter_InterruptEnable` respectively.

Usage

Use the following steps to integrate and use the driver in the application:

Figure 12: Flowchart for Operation



X25561-070921

1. Include the driver header files `xv_warp_init_gen_l2.h` and `xv_warp_filter_l2.h`.
2. Declare an instance of the Warp Initializer type `XV_warp_init WarpInitInst`, and an instance of the Filter type `XV_warp_filter WarpInst`.

3. Initialize the Warp Initializer and Warp Filter instance at power on:

```
int XV_warp_ Initialize (XV_warp_init *InstancePtr, u16 DeviceId);
```

```
int XV_warp_filter_Initialize (XV_warp_filter *InstancePtr, u16 DeviceId);
```

This function accesses the hardware configuration and initializes the core instance structure.

4. The application must register the ISRs for both the cores with the system interrupt controller and set the application callback function. This function is called by the driver when the frame done IRQ is triggered.

5. Enable the Warp Initializer IP interrupt:

```
void XVWarpInit_EnableInterrupts(XV_warp_init *InstancePtr, u32 Mask);
```

6. Fill the Warp Initializer descriptor and start the IP:

```
int XVWarpInit_ProgramDescriptor(XV_warp_init *InstancePtr, u32 Descnum, XV_warp_init_InputConfigs *ConfigPtr);
```

```
int XVWarpInit_start_with_desc(XV_warp_init *InstancePtr, u32 descnum);
```

7. Wait for the interrupt.

8. Enable the Warp Filter IP interrupt:

```
void XVWarpFilter_InterruptEnable(XV_warp_filter *InstancePtr, u32 Mask);
```

9. Fill the Warp Filter descriptor and start the IP:

```
void XVWarpFilter_ProgramDescriptor(XV_warp_filter *InstancePtr, u32 DescNum, XV_warp_filter_InputConfigs *configPtr, U32 valid_seg, U32 lblock_count, U32 line_num);
```

```
void XVWarpFilter_Start(XV_warp_filter *InstancePtr);
```

10. Wait for the interrupt and update the buffer addresses of the next frame.

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to <https://docs.amd.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

References

These documents provide supplemental material useful with this guide:

1. *Video Frame Buffer Read and Video Frame Buffer Write LogiCORE IP Product Guide* ([PG278](#))
 2. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
 3. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
 4. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
 5. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
 6. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
 7. *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#))
 8. *Vivado Design Suite: AXI Reference Guide* ([UG1037](#))
 9. *Vitis High-Level Synthesis User Guide* ([UG1399](#))
-

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
07/09/2025 Version 1.1	
Features	Updated the section.
Obtaining the .elf File	Updated steps and screenshots.
07/01/2024 Version 1.1	
Features	Updated the section.
Warp Initializer IP Descriptor Structure	Added a note.
12/19/2023 Version 1.1	
Warp Initializer IP Descriptor Structure	Updated descriptor structure.
Warp Filter Descriptor Structure	Updated descriptor structure.

Section	Revision Summary
05/11/2022 Version 1.1	
Chapter 3: Product Specification	- Added new fields in Table 5: Warp Initializer Descriptor Fields - Added a new register in Table 4: Register Address Space for Warp Initializer - Updated Control (0x00000) Register in Register Address Space for Warp Filter
12/07/2021 Version 1.0	
Obtaining the .elf File	General updates in the project settings.
07/15/2021 Version 1.0	
Initial release.	N/A

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING

OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2021–2025 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Kintex, UltraScale, UltraScale+, Versal, Vitis, Vivado, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.